Knowledge Distillation and Deployment of a Lightweight Command-Parsing LM on a Local Cloud

Name: Ritika
Roll number: 230101067
Course: CS3081 Cloud Computing
Repository: `https://github.com/<your-username>/230101067_CloudComputing_Assignment`

1. Architecture and design decisions

The project is a local cloud with four layers. A k3d cluster is created from `cluster/k3d.yaml` (one server, one agent, Traefik and metrics-server kept enabled). SeaweedFS runs in namespace `data` and exposes an S3-compatible API. A parallel Kubernetes Indexed Job in namespace `pipeline` reads `datasets/commands_pool.jsonl`, labels each command with a host-run Ollama `phi3.5`, and writes one raw shard object per worker. A cleaning step validates the replies and writes `datasets/v1/train.jsonl` with a SHA-256 checksum. Training distils the teacher replies into `EleutherAI/pythia-70m` and stores `model.tar.gz` and `model.sha256` in the `models` bucket. The FastAPI service runs in namespace `app`, downloads the model at start-up, and is exposed by a Traefik Ingress.

The main design choice is to keep weights out of the image and out of Git. The service image contains only Python and pinned libraries; `/readyz` stays 503 until the model has been downloaded, checksum-verified, extracted, and loaded. This makes cold start observable and makes a model update a change of `MODEL_URI`/`MODEL_VERSION`, not a rebuild. Ollama runs on the host rather than in the cluster because the teacher is only needed during Task 1 and host networking avoids a multi-gigabyte GPU image. The workers reach it through `host.k3d.internal:11434`. Every component is stateless except SeaweedFS, so Pod deletion and rolling updates are safe.

Version pins live in `training/config.yaml`, `service/requirements.txt`, `worker/requirements.txt` and `training/requirements.txt`. All cluster configuration is in `k8s/`; the only host-generated state is the git-ignored `.env.local` containing S3 credentials.

2. Task 1: object storage and parallel teacher labelling

2.1 Storage layout

SeaweedFS is deployed by `k8s/seaweedfs.yaml` into namespace `data`. Two buckets are created: `datasets` and `models`. `scripts/init_storage.py` up-

loads the provided `commands_pool.jsonl` to `datasets/commands_pool.jsonl`. `prompts/teacher_prompt.txt` asks for exactly the four keys and the eight allowed intents, gives three examples, and is fixed at `temperature=0`, `seed=42`, `top_k=1`.

2.2 Sharding and resumability

`worker/worker.py` reads `SHARD_INDEX` and `SHARD_COUNT`, keeps the command on line `i` when `i % SHARD_COUNT == SHARD_INDEX`, and writes `datasets/teacher/shard-<index>.jsonl` with exactly `{"id","text","teacher_raw"}`. The Kubernetes Job uses `completionMode: Indexed`, `completions: 4`, `parallelism: 2`; Kubernetes supplies `JOB_COMPLETION_INDEX`, which the worker exports as `SHARD_INDEX`. With four completion indexes, each of the 3,000 line numbers is selected by exactly one worker, so there are no gaps or repeats.

Resume is built in: before labelling, the worker reads its own existing shard object into a set of completed ids and skips them. Progress is written after every reply by default (atomic temp-file + rename), so an interrupted multi-hour run loses at most one command. The cleaner also refuses to proceed if ids are repeated across shards; `--allow-incomplete` is only for smoke runs.

2.3 Cleaning rules

`worker/clean.py` applies rejection precedence (a) -> (b) -> (c) -> (d) and counts each record exactly once:

- (a) `teacher_raw` is not valid JSON or is not an object;
- (b) keys are not exactly `intent`, `date`, `time`, `place`;
- (c) `intent` is not one of the eight allowed values;
- (d) a slot is neither null nor a string that appears in the command (case-insensitive, whitespace-normalised);
- duplicates are removed by normalised command text, keeping the smallest id.

Survivors are sorted by numeric id and written to `datasets/v1/train.jsonl`; the SHA-256 line is written to `datasets/v1/train.sha256`. The accounting invariant below is asserted in code:

```
input = invalid_json + wrong_keys + bad_intent + bad_slot + duplicates + kept
```

```
<FILL: Table 1 - Task 1 counts for the smoke run and the full run
if available>
```

2.4 Sharding rationale

Four shards are the smallest count that gives two waves at `parallelism: 2`, so the cluster exercises both parallel workers and resume without creating idle pods. Sharding by line number, rather than by id, is exactly what the assignment specifies and is independent of the order of ids. Because every worker owns a disjoint residue class, the union is complete without a merge step. The cost is that a worker's commands are interleaved through the pool, but the worker sorts its own output by id, so each object is deterministic. Four workers also mean four objects, which is small enough to inspect by hand and large enough to show that the cleaning step really reads multiple shards. The local smoke run used the same code with four shards and one worker at a time, for 50 commands per shard (200 total), because a full CPU teacher run of 3,000 commands takes several hours; the Kubernetes Job is the intended full path and is resumable.

3. Task 2: training the student

3.1 Configuration

`training/config.yaml` records the six required items:

Item	Value
Random seed	42 (`torch.manual_seed`, NumPy, Python `random`; fixed in code)
Learning rate	5e-5
Batch size	4 (gradient accumulation 4, effective 16)
Epochs	2
Maximum sequence length	256
Library versions	Python 3.12, torch 2.4.1, transformers 4.44.2, tokenizers 0.19.1, safetensors 0.4.5, huggingface-hub 0.24.6, numpy 1.26.4, PyYAML 6.0.2, boto3 1.35.24

`training/config_v2.yaml` changes only the seed (43), epochs (3), version and loss file. `make train` reads only `training/config.yaml`; `make train SUBSET=50` reads the same file and caps the rows. `training/loss.csv` is written and flushed after every epoch.

3.2 Example format and loss

Each record becomes `Command: <text>\nJSON: <teacher reply>`. The prompt tokens are masked with `-100`, so the cross-entropy is computed

only on the teacher reply tokens plus EOS; this matches "the model learns to produce the teacher's reply, given the command" and prevents the 70M model from spending capacity on reproducing the prompt. The model is `EleutherAI/pythia-70m`, loaded in float32, trained with AdamW and a linear warm-up schedule on CPU. The target is sequence-level: the exact teacher string, not logits.

`<FILL: Table 2 - v1 and v2 loss per epoch, records used, wall time>`

3.3 Evaluation

Predictions are generated for the teacher (`phi3.5`, same prompt, temperature 0, seed 42), untrained Pythia-70M, and the two students. Raw outputs and score reports are in `results/`. The official `tools/score.py` computes JSON validity, intent accuracy, slot accuracies and full-frame accuracy.

`<FILL: Table 3 - teacher/base/v1/v2 scores>`

`<FILL: short discussion of what changed>`

3.4 Sequence-level versus logit-based distillation

Sequence-level distillation trains the student on the teacher's generated sequence, treating it as a hard target. The loss is the ordinary next-token cross-entropy on that sequence. Logit-based distillation trains the student to match the teacher's full next-token probability distribution at each position, typically with a temperature-softened KL divergence. Logit-based distillation is not possible here because the teacher is served by Ollama, which exposes only the generated text through `/api/generate`; it does not return per-token logits. Even if it did, the teacher and student use different tokenizers, so their vocabularies would need an alignment step. A limitation of the sequence-level approach is that it discards the teacher's uncertainty and cannot teach the student about near-miss alternatives; with only about 1,500 examples, the student can also memorise surface forms instead of generalising to unseen commands.

4. Task 3: inference service and container

The service is a FastAPI application on port 8000 with `POST /parse`, `GET /healthz` and `GET /readyz`. `/parse` returns only the four-key object and sets `X-Model-Version` and `X-Served-By` (the Pod name from a downward-API environment variable, with `HOSTNAME` and `socket.gethostname()` as fallbacks). The output validator checks exact keys, the eight intents, and string-or-null slots. If the model output is not valid, the service returns HTTP 500 with exactly `{"error":"invalid_model_output"}`; it never returns malformed JSON

with 200. FAULT_INJECT=1 replaces the generated text with not json before validation, which exercises that path.

At start-up the service downloads s3://models/<version>/model.tar.gz using boto3 against S3_ENDPOINT, verifies model.sha256 if present, safely extracts the archive, and loads the model and tokenizer. MODEL_URI, MODEL_VERSION, S3_ENDPOINT and MAX_NEW_TOKENS come from the environment; in Kubernetes the two S3 credentials come only from the model-store-credentials Secret. The image is built by service/Dockerfile from python:3.12-slim, installs the CPU-only PyTorch wheel and pinned dependencies, and copies no weights. It runs as UID 10001 with a read-only root filesystem and a writable /tmp emptyDir for the model cache.

<FILL: local 50-request result: p50/p95, peak RSS, success count, load time>

The 50-request test was run against the local process (no Docker daemon is available in this build environment). The container-level 1 CPU / 1 GiB test is therefore marked as not executed; the commands are python scripts/bench_local.py --url ... --n 50 --pid <pid> and the Docker invocation in README.md. The service is single-worker and generation is serialised by a lock, so peak RSS is stable rather than accumulating.

5. Task 4: Kubernetes deployment, scaling and reliability

k8s/app.yaml creates a two-replica Deployment in app with CPU request 250m/limit 1, memory request 512Mi/limit 1Gi, liveness /healthz and readiness /readyz, maxUnavailable: 0, maxSurge: 1, a non-root read-only filesystem, and the Secret-backed S3 credentials. k8s/ingress.yaml routes /parse, /healthz and /readyz through Traefik; k8s/hpa.yaml scales 1-2 replicas at 70% CPU with a 60-second scale-down window.

<FILL: cluster status. In this environment Docker/k3d are not installed, so the following are NOT MEASURED: HPA 1->2->1, rolling update failure %, rollback header, pod-deletion recovery, three cold starts and median, per-replica request share. The exact commands are in docs/RUNBOOK.md.>

Local limitation: the cluster is two k3d nodes on one laptop and shares one CPU and one disk. Scaling to two replicas demonstrates the mechanism but does not provide machine-level fault tolerance or a realistic multi-node network. A real limitation of the HPA is that CPU utilisation includes model download and start-up work; a pod can be scaled before it is Ready, and the readiness probe is what protects the Service from traffic. During a rolling update the old pod remains Ready until the new pod downloads and loads the model, so the update can take as long as a cold start.

6. Task 5: subnetting and NACL

Requirements are allocated largest first, each immediately after the previous block, with the smallest prefix that fits:

Subnet	Network	Prefix	Usable host range	Broadcast	Usable
app	10.20.0.0	/22	10.20.0.1-10.20.3.254	10.20.3.255	1022
public	10.20.4.0	/24	10.20.4.1-10.20.4.254	10.20.4.255	254
data	10.20.5.0	/25	10.20.5.1-10.20.5.126	10.20.5.127	126
management	10.20.5.128	/27	10.20.5.129-10.20.5.158	10.20.5.159	30

The app subnet needs 1,000 hosts, so /22 (1,022 usable) is the smallest fit; public needs 250, so /24 (254); data needs 100, so /25 (126); management needs 20, so /27 (30). The total consumed is 10.20.0.0/22, 10.20.4.0/24 and the two halves of 10.20.5.0/24, leaving space for growth.

`network/nacl.json` is the public-subnet NACL. Inbound rule 100 denies 198.51.100.0/24 for all protocols and ports before the web allows; rule 110 allows TCP 80, rule 120 allows TCP 443, rule 130 allows TCP 22 only from 10.20.5.128/27, and rule 140 allows return traffic from 10.20.0.0/22 on ephemeral ports 1024-65535. Outbound rule 100 allows TCP 8000 to 10.20.0.0/22, and rule 110 allows replies to clients on ports 1024-65535; there is an implicit deny for everything else. The official simulator reports **6 / 6 = 100.0%**.

A NACL is stateless: it does not remember that an inbound connection created a flow, so the response packets are evaluated as new packets and need their own rules in both directions. Rule order matters because NACL rules are evaluated from the lowest number and the first match wins; the deny for 198.51.100.0/24 must be numbered below the TCP 80/443 allows, otherwise those allow rules would match first and the abusive range would be allowed.

7. Task 6: reproducibility, testing and CI/CD

`make up` creates the cluster, secrets, SeaweedFS, buckets, images, uploads the model and applies the Deployment/Service/Ingress/HPA; `make down` deletes the cluster. From a fresh clone the whole system is two commands: `make install-ml` and `make up`, or just `make up` because the target bootstraps the virtual environment if it is missing. `make up` does not commit credentials:

scripts/create_secrets.sh generates them, writes them to the git-ignored .env.local with mode 0600, and creates the Kubernetes Secrets.

The test suite has 49 tests covering the cleaner's rejection precedence and duplicate rule, accounting invariant, sharding and resume, subnet arithmetic, the official NACL simulator, service API and exact error body, readiness before load, output validation, storage backends, and cost arithmetic. ruff check . passes, pytest -q reports 49 passed, and gitleaks detect --no-git reports no leaks found (evidence in results/).

The CI workflow in .github/workflows/ci.yml runs in the required order: lint (ruff) -> unit-tests (pytest) -> model-evaluation (downloads model.tar.gz and model.sha256 from the final GitHub Release, verifies the checksum, runs training/predict.py, then tools/score.py) -> quality-gate (--min-validity 95). A separate secret-scan job runs gitleaks. A manual workflow_dispatch input bad_model=true runs the untrained Pythia-70M through the same gate and is expected to fail; its URL must be inserted here after the repository is pushed:

<FILL: link to the failing quality-gate run>

AI_USE.md lists the AI tools and the checks applied; every generated file was reviewed, linted, and tested, and the measured results come from commands in results/, not from the assistant.

8. Task 7: performance, cost and review

<FILL: local Locust results: p50, p95, throughput, error rate, total requests; note that this is a local process, not the cluster>

The cluster measurements (kubectl top pods, cold-start median, per-replica share) are not available in this build environment; the exact collection commands are in docs/RUNBOOK.md. They must be filled from a run on a Docker host before submission.

8.1 Cost

A replica is billed for its CPU limit and memory limit for every hour it runs, so with two replicas at 1 vCPU and 1 GiB each:

```
per replica per hour = 1 * 3.50 + 1 * 0.45 = 3.95 INR
all replicas per hour = 2 * 3.95 = 7.90 INR
requests per hour = measured requests/second * 3600
cost per 1,000 = (7.90 / requests per hour) * 1000
```

<FILL: exact arithmetic with measured throughput and final INR value>

`scripts/cost.py` performs exactly this calculation from `results/loadtest_stats.csv` and `data/rate_card.json`, and writes `results/cost.json`.

8.2 SLO

SLO: **99.5% of `/parse` requests succeed (HTTP 200) over one load-test window**. For the measured window with `N` requests, the error budget is `floor(0.005 * N)` failed requests. `<FILL: N, allowed failures, actual failures, budget used/remaining, met/not met>`

8.3 Review

Component or concern	Evidence (measured)	Limitation found	Improvement proposed
Data and storage	`<FILL>`	`<FILL>`	`<FILL>`
Model quality	`<FILL>`	`<FILL>`	`<FILL>`
Latency and throughput	`<FILL>`	`<FILL>`	`<FILL>`
Resource utilisation	`<FILL>`	`<FILL>`	`<FILL>`
Reliability and recovery	`<FILL>`	`<FILL>`	`<FILL>`
Security and cost	`<FILL>`	`<FILL>`	`<FILL>`

9. Limitations and improvements

The largest limitation of this submission is environmental: the machine used to build it has no Docker daemon and no k3d/kubectl, and its free memory is about 1.5 GiB while `phi3.5` needs about 2.6 GiB. The code, manifests and tests are complete, but the cluster experiments and the full 3,000-command teacher run could not be executed here. The report marks those values as not measured rather than inventing them. The next step on a Docker host is to run `make up`, `make teacher-label`, `make clean`, `make train`, `make train-v2`, `make evaluate`, then the Task 4 runbook and `make loadtest`; each command writes its evidence to `results/`.

Beyond the environment, a 70M-parameter student will not match a 3.8B teacher; the smoke training data is much smaller than the required 1,500 records, so the local scores are only a pipeline check. The HPA reacts to CPU rather than to queue depth, the service serialises generation under a lock, and the k3d cluster is two nodes on one machine. Better choices would be a queue-depth or latency-based autoscaler, a batched/streaming inference server, and a multi-machine cluster to test real node failure. On the data side,

more teacher examples, stricter slot grounding, and a held-out split would reduce overfitting. On the security side, the S3 credentials are generated and Secret-backed, the image is non-root and read-only, and the NACL is default deny; a network policy restricting the app namespace to SeaweedFS and the Ingress would add defence in depth.